

图上的结构算法

不要背七段代码：追踪候选域、提交时刻与仍然成立的事实

408 · 数据结构 Topic DS-08

母问题：图上的“下一步”凭什么不会破坏全局目标？

所有算法先压缩为同一条生成链：

Goal → Candidate Domain → Local Action → Commit Point → Invariant → Global Result

最容易混淆的不是循环语法，而是“候选对象是谁、何时可以永久确定”。Prim 与 Kruskal 都求最小生成树，却分别在单棵树边界与全图森林之间选择；Dijkstra、Bellman–Ford 与 Floyd 都做松弛，却分别固定顶点、路径边数上界与允许经过的中间点集合。

本册定位与 Owner 边界

- **Owns**：Prim/Kruskal、Dijkstra/Bellman–Ford/Floyd、拓扑排序与 AOE 关键路径的目标、不变量、提交条件和失败证据。
- **Uses**：DS07 提供图表示与遍历状态；Kruskal 直接复用 DS06 Union-Find；Prim/Dijkstra 复用 DS05 的“按优先级取候选”抽象契约。
- **接口事实**：当前 Prim 用数组扫描，Dijkstra 用标准库最小优先队列，并未直接复用 DS05 的整数最大堆类；不能把概念依赖写成代码依赖。
- **Stops**：堆、并查集和图存储内部机制不在本册重讲；工程路由协议只可调用最短路模型。

目录

1	先认候选域，再认算法名	3
2	最小生成树：同一目标，两种生长几何	3
2.1	Prim：一棵树向外吞并	3
2.2	Kruskal：多棵树逐步合并	4
3	最短路：同一个松弛式，三种“已经解决”	5
3.1	Dijkstra：固定当前最近的顶点	5
3.2	Bellman–Ford：固定路径允许使用的边数	7
3.3	Floyd：固定允许经过的中间点集合	8
4	DAG：从释放约束到识别零时差活动	9
4.1	拓扑排序：删除的不是点，而是前置约束	9
4.2	DAG 表达式与共享子问题	9
4.3	关键路径：正向求最早，反向求最迟	10
5	测试不是报喜，而是攻击不变量	12

6	复杂度与适用边界	14
7	考场与代码复原协议	14

1 先认候选域，再认算法名

算法	目标对象	候选域	提交时刻	失败证据
Prim	一棵 MST	当前树到树外的割边	树外顶点以最小跨边权加入	仍有顶点却无有限割边
Kruskal	一棵 MST	全图尚未处理的边	最小边连接两个不同分量	处理完仍不足 $V - 1$ 条边
Dijkstra	单源最短路	未过期的暂定距离	最小暂定距离出队	出现负权边
Bellman-Ford	单源最短路	每一轮的全部有向边	第 i 轮保证至多 i 条边的最短路	$V - 1$ 轮后仍可松弛
Floyd	全源最短路	所有点对 (i, j)	中间点集合由前 $k - 1$ 个扩为前 k 个	最终存在 $d[i][i] < 0$
拓扑排序	约束合法序列	当前入度为零的顶点	顶点出队并释放后继约束	输出顶点数小于 V
关键路径	AOE 最长工期与零时差活动	拓扑序上的事件和活动	正向、反向两遍完成后	图有环或工期为负判时差

统一记忆锚点

不要记“某算法总是取最小”。先补完整句子：从哪个候选域，按什么量取谁；取出后是暂时用于传播，还是从此永久确定？只要候选域或提交时刻答错，代码即使长得像模板，证明也已经失效。

2 最小生成树：同一目标，两种生长几何

2.1 Prim：一棵树向外吞并

设 S 为已经进入树的顶点集合， $\text{best}[v]$ 表示从 S 跨到顶点 v 的最轻边权。每轮选择树外 best 最小的顶点加入 S ，再用它的邻边缩小其他顶点的 best 。

循环不变量

已提交顶点始终连成一棵树；每个未提交顶点的 best 是当前割 $(S, V - S)$ 上到该点的最轻已知边。割性质保证本轮最轻跨边是安全边。若树外仍有顶点而所有 best 都是无穷大，证据不是“程序没找到”，而是图不连通。

Prim 的初始化不是细节：任选起点令其 $\text{best} = 0$ ，其余顶点为 ∞ ；每次提交顶点时记录使其进入树的 parent 边。若图有并列最轻跨边，不同 tie-break 可能得到不同但同权的 MST；若题目问唯一性，应检查是否存在某个割上的并列最轻边（或给出足以保证所有边权互异的条件）。数组扫描版每轮寻找最小未提交顶点，复杂度为 $O(V^2 + E)$ ；堆版可降为 $O((V + E) \log V)$ ，但两者的割不变量相同。

```

1 inline Distance prim_weight(std::size_t vertex_count, const std::vector<Edge>&
   edges) {
2     if (vertex_count == 0) {
3         return 0;
4     }
5     std::vector<std::vector<std::pair<std::size_t, int>>> graph(vertex_count);
6     for (const Edge& edge : edges) {

```

```

7     check_edge(vertex_count, edge);
8     graph[edge.from].push_back({edge.to, edge.weight});
9     graph[edge.to].push_back({edge.from, edge.weight});
10 }
11
12 std::vector<Distance> best_crossing_edge(vertex_count, kInfinity);
13 std::vector<bool> fixed(vertex_count, false);
14 best_crossing_edge[0] = 0;
15 Distance total = 0;
16 for (std::size_t step = 0; step < vertex_count; ++step) {
17     std::size_t chosen = vertex_count;
18     for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
19         if (!fixed[vertex] &&
20             (chosen == vertex_count || best_crossing_edge[vertex] <
21              best_crossing_edge[chosen])) {
22             chosen = vertex;
23         }
24     }
25     if (chosen == vertex_count || best_crossing_edge[chosen] == kInfinity) {
26         throw std::invalid_argument("graph is disconnected");
27     }
28     fixed[chosen] = true;
29     total += best_crossing_edge[chosen];
30     for (const auto [next, weight] : graph[chosen]) {
31         if (!fixed[next] && weight < best_crossing_edge[next]) {
32             best_crossing_edge[next] = weight;
33         }
34     }
35 }
36 return total;
37 }

```

代码清单 1: Prim: 候选域是当前树的边界

2.2 Kruskal: 多棵树逐步合并

Kruskal 不维护“当前树”，而维护无环森林。边按权从小到大扫描；只有两端属于不同连通分量时，才把边提交并合并分量。

同一张图为什么会产生两种轨迹

边为 01(1), 12(2), 23(1), 03(8)。从顶点 0 出发的 Prim 轨迹是

$$\{0\} \rightarrow \{0, 1\} \rightarrow \{0, 1, 2\} \rightarrow \{0, 1, 2, 3\};$$

Kruskal 则可先同时形成分量 $\{0, 1\}$ 与 $\{2, 3\}$ ，再用 12(2) 合并。两者总权都为 4，但状态几何完全不同：Prim 是单树扩张，Kruskal 是森林合并。

```

1 inline Distance kruskal_weight(std::size_t vertex_count, std::vector<Edge>
   edges) {
2     for (const Edge& edge : edges) {
3         check_edge(vertex_count, edge);
4     }

```

```

5  std::sort(edges.begin(), edges.end(),
6      [](const Edge& left, const Edge& right) { return left.weight <
7          right.weight; });
8  ds06::UnionFind components(vertex_count);
9  Distance total = 0;
10 std::size_t chosen_edges = 0;
11 for (const Edge& edge : edges) {
12     if (components.unite(edge.from, edge.to)) {
13         total += edge.weight;
14         ++chosen_edges;
15         if (chosen_edges + 1 == vertex_count) {
16             break;
17         }
18     }
19 }
20 if (vertex_count > 0 && chosen_edges + 1 != vertex_count) {
21     throw std::invalid_argument("graph is disconnected");
22 }
23 return total;
24 }

```

代码清单 2: Kruskal: Union-Find 维护森林分量

MST 的边界与并列权值

Prim 与 Kruskal 的安全性来自割性质和环性质，而不是来自“每次看起来最小”。并列权值时具体边集可能不同，但只要图连通，所得树的总权值相同；若题目要求唯一的边集，必须再检查权值是否互异或给出 tie-break 规则。非连通图不能输出一棵覆盖全部顶点的 MST，只能得到最小生成森林，程序应把缺少 $V - 1$ 条提交边作为可解释失败。

3 最短路：同一个松弛式，三种“已经解决”

对边 (u, v, w) ，三种算法共享局部动作

$$d[v] \leftarrow \min\{d[v], d[u] + w\},$$

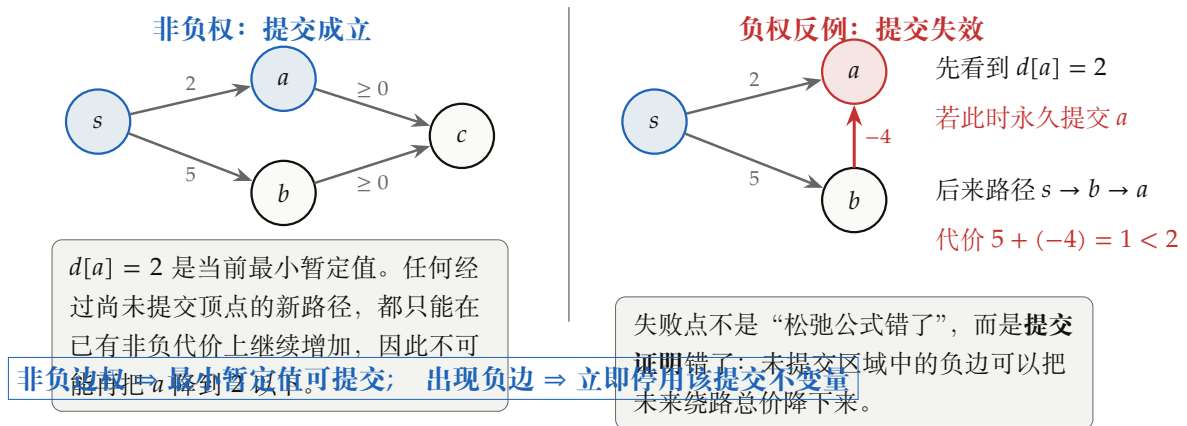
但局部动作相同不代表外层循环可以互换。外层循环决定每一步之后究竟固定了什么。

3.1 Dijkstra: 固定当前最近的顶点

非负权保证未来绕路只会继续增加长度。因此，当未过期的最小暂定距离 (d, u) 出队时，不存在一条经未确定顶点绕回来使 $d[u]$ 更小的路径。

Dijkstra: 非负权让“当前最小暂定距离”可以永久提交

左：非负权下，未提交区域再绕一圈只会增加代价；右：一条负边即可让已经提交的距离被后来路径改小。



右侧反例只改变一条边为负权：松弛动作本身仍合法，真正失效的是“当前最小暂定距离可以永久提交”的证明。

读代码时只盯三个状态

1. `distance[v]`：目前最好上界，不等于所有时刻都已最终确定；
2. `pending`：可能含同一顶点的旧候选；
3. `current_distance != distance[current]`：丢弃过期候选，只有未过期最小项才承担提交语义。

负权边会破坏“取出即确定”的证明，所以实现必须在运行前拒绝它，而不是算完再碰碰运气。

初始化时仅源点距离为 0，其余为 ∞ ，前驱为空；松弛前必须确认 $d[u] \neq \infty$ ，并用足够宽的整数承载加法。若采用邻接矩阵，扫描每个未确定顶点的全部候选邻居；若采用优先队列，允许同一顶点出现多个候选副本，但必须用“出队距离是否仍等于当前 `distance`”过滤旧项。路径恢复沿 `predecessor` 反向走到源点再逆序；不可达点没有合法路径，不能把 ∞ 当作可加的数。

```

1 inline std::vector<Distance> dijkstra(std::size_t vertex_count,
2                                       const std::vector<Edge>& edges,
3                                       std::size_t source) {
4     if (source >= vertex_count) {
5         throw std::out_of_range("source vertex out of range");
6     }
7     std::vector<std::vector<std::pair<std::size_t, int>>> graph(vertex_count);
8     for (const Edge& edge : edges) {
9         check_edge(vertex_count, edge);
10        if (edge.weight < 0) {
11            throw std::invalid_argument("dijkstra requires nonnegative edges");
12        }
13        graph[edge.from].push_back({edge.to, edge.weight});
14    }
15
16    using Candidate = std::pair<Distance, std::size_t>;
17    std::priority_queue<Candidate, std::vector<Candidate>, std::greater<
18        Candidate>> pending;
19    std::vector<Distance> distance(vertex_count, kInfinity);

```

```

19 distance[source] = 0;
20 pending.push({0, source});
21 while (!pending.empty()) {
22     const auto [current_distance, current] = pending.top();
23     pending.pop();
24     if (current_distance != distance[current]) {
25         continue;
26     }
27     for (const auto [next, weight] : graph[current]) {
28         const Distance candidate = current_distance + static_cast<Distance>(
weight);
29         if (candidate < distance[next]) {
30             distance[next] = candidate;
31             pending.push({candidate, next});
32         }
33     }
34 }
35 return distance;
36 }

```

代码清单 3: Dijkstra: 非负权、最小候选与过期项过滤

3.2 Bellman–Ford: 固定路径允许使用的边数

第 i 轮扫描全部边后, 可以保证所有“至多含 i 条边”的最短路已被传播。没有可达负环时, 简单最短路至多含 $V - 1$ 条边; 因此 $V - 1$ 轮后若仍能松弛, 就得到一个可达负环证据。

最常漏掉的保护条件

若 $d[u] = \infty$, 边 $u \rightarrow v$ 不可参与加法, 更不能据此更新 v 。“无穷大不参与算术”同时是语义规则和防溢出规则。

```

1 inline std::vector<Distance> bellman_ford(std::size_t vertex_count,
2                                           const std::vector<Edge>& edges,
3                                           std::size_t source) {
4     if (source >= vertex_count) {
5         throw std::out_of_range("source vertex out of range");
6     }
7     for (const Edge& edge : edges) {
8         check_edge(vertex_count, edge);
9     }
10    std::vector<Distance> distance(vertex_count, kInfinity);
11    distance[source] = 0;
12    for (std::size_t round = 1; round < vertex_count; ++round) {
13        bool changed = false;
14        for (const Edge& edge : edges) {
15            if (distance[edge.from] == kInfinity) {
16                continue;
17            }
18            const Distance candidate = distance[edge.from] + static_cast<Distance>(
edge.weight);
19            if (candidate < distance[edge.to]) {

```

```

20     distance[edge.to] = candidate;
21     changed = true;
22 }
23 }
24 if (!changed) {
25     break;
26 }
27 }
28 for (const Edge& edge : edges) {
29     if (distance[edge.from] != kInfinity &&
30         distance[edge.from] + static_cast<Distance>(edge.weight) < distance[
31         edge.to]) {
32         throw std::invalid_argument("reachable negative cycle");
33     }
34 }
35 return distance;
36 }

```

代码清单 4: Bellman-Ford: 轮数就是路径边数上界

3.3 Floyd: 固定允许经过的中间点集合

令 $D^{(k)}[i][j]$ 表示只允许前 k 个顶点作为中间点时, i 到 j 的最短距离, 则

$$D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]).$$

因此 middle 必须放在最外层: 完成第 k 层之后, 整张矩阵才具有统一的“允许中间点集合”语义。

若需要恢复路径, 通常维护 $next[i][j]$ 或 $path[i][j]$: 当经 k 松弛成功时, 令 $next[i][j] = next[i][k]$ (或令前驱变为 $path[k][j]$), 而不是把中间点 k 直接当作下一跳。初始化时 $next[i][j] = j$ 仅对存在直接边的点对成立; 无边点对保持空标记。最后若 $D[i][i] < 0$, 说明存在可达负权回路, 相关最短路不是有限值; 若只有负权边而无负环, Floyd 仍可正确处理。

```

1 inline std::vector<std::vector<Distance>> floyd_warshall(
2     std::size_t vertex_count, const std::vector<Edge>& edges) {
3     std::vector<std::vector<Distance>> distance(
4         vertex_count, std::vector<Distance>(vertex_count, kInfinity));
5     for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
6         distance[vertex][vertex] = 0;
7     }
8     for (const Edge& edge : edges) {
9         check_edge(vertex_count, edge);
10        distance[edge.from][edge.to] =
11            std::min(distance[edge.from][edge.to], static_cast<Distance>(edge.
12            weight));
13    }
14    for (std::size_t middle = 0; middle < vertex_count; ++middle) {
15        for (std::size_t from = 0; from < vertex_count; ++from) {
16            if (distance[from][middle] == kInfinity) {
17                continue;
18            }
19            for (std::size_t to = 0; to < vertex_count; ++to) {

```

```

19         if (distance[middle][to] == kInfinity) {
20             continue;
21         }
22         distance[from][to] = std::min(
23             distance[from][to], distance[from][middle] + distance[middle][to])
24     ;
25     }
26 }
27 for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
28     if (distance[vertex][vertex] < 0) {
29         throw std::invalid_argument("graph contains a negative cycle");
30     }
31 }
32 return distance;
33 }

```

代码清单 5: Floyd: 最外层循环扩张允许中间点集合

三算法一眼分流

- **Dijkstra**: 固定顶点; 需要非负权; 适合单源。
- **Bellman-Ford**: 固定路径边数上界; 允许负边; 可检测源点可达负环。
- **Floyd**: 固定允许的中间点集合; 处理所有点对; 对角线负值暴露负环。

看到“松弛”还不能选算法, 必须再问: 题目要求固定哪一种状态?

4 DAG: 从释放约束到识别零时差活动

4.1 拓扑排序: 删除的不是点, 而是前置约束

入度为零表示一个顶点的全部前置约束已经释放。顶点出队后, 对每条出边将后继入度减一; 只有减到零的后继才进入候选队列。若队列耗尽但仍有顶点未输出, 这些顶点之间存在无法释放的循环依赖。

若某一步同时有两个或以上入度为零的候选, 拓扑序不唯一; 若每一步都恰有一个候选且最终输出 V 个顶点, 序列才唯一。这个判据直接来自 Kahn 的候选域, 不需要枚举所有排列。DFS 版本可用三色状态: 灰色节点再次遇到即为后向边和环, 只有从后继全部完成后才把顶点压入结果栈, 最后逆序得到拓扑序; 不能把“DFS 访问顺序”直接当作拓扑序。

Kahn 算法中若把入度为零的候选放入栈、队列或优先队列, 会得到不同的合法拓扑序: FIFO 倾向于较早发现的顶点, 栈产生另一种深度偏序, 最小堆可得到字典序最小序列。选择容器改变的是 tie-break, 不改变“出队后删除其出边约束”的正确性。拓扑排序完成后应验证输出长度是否为 V ; 少于 V 时剩余顶点的入度都无法降到零, 环是可解释失败证据。

4.2 DAG 表达式与共享子问题

将表达式表示为 DAG 时, 叶节点是操作数, 内部节点是运算, 边表示数据依赖。相同子表达式可共享一个节点, 因而图的节点数不必等于表达式树展开后的节点数; 求值必须按拓扑序, 任何节点只有在所有前驱值已就绪后才能计算。若题目要求把 DAG 还原为中缀表达式, 需要在分叉处明确括号和共享节点的重复打印策略; “拓扑序”只保证依赖合法, 不保证算术运算符优先级。

```

1 inline std::vector<std::size_t> topological_order(
2     std::size_t vertex_count, const std::vector<Edge>& edges) {
3     std::vector<std::vector<std::size_t>> graph(vertex_count);
4     std::vector<std::size_t> indegree(vertex_count, 0);
5     for (const Edge& edge : edges) {
6         check_edge(vertex_count, edge);
7         graph[edge.from].push_back(edge.to);
8         ++indegree[edge.to];
9     }
10    std::queue<std::size_t> ready;
11    for (std::size_t vertex = 0; vertex < vertex_count; ++vertex) {
12        if (indegree[vertex] == 0) {
13            ready.push(vertex);
14        }
15    }
16    std::vector<std::size_t> order;
17    while (!ready.empty()) {
18        const std::size_t current = ready.front();
19        ready.pop();
20        order.push_back(current);
21        for (const std::size_t next : graph[current]) {
22            if (--indegree[next] == 0) {
23                ready.push(next);
24            }
25        }
26    }
27    if (order.size() != vertex_count) {
28        throw std::invalid_argument("directed graph contains a cycle");
29    }
30    return order;
31 }

```

代码清单 6: Kahn 拓扑排序：候选域是当前零入度点

4.3 关键路径：正向求最早，反向求最迟

AOE 网中，顶点是事件，边是有持续时间的活动。沿拓扑序正向做最长路松弛：

$$ve[v] = \max_{u \rightarrow v} \{ve[u] + w(u, v)\}.$$

项目工期是所有事件最早发生时间的最大值。再以工期初始化 vl ，沿逆拓扑序反向收紧：

$$vl[u] = \min_{u \rightarrow v} \{vl[v] - w(u, v)\}.$$

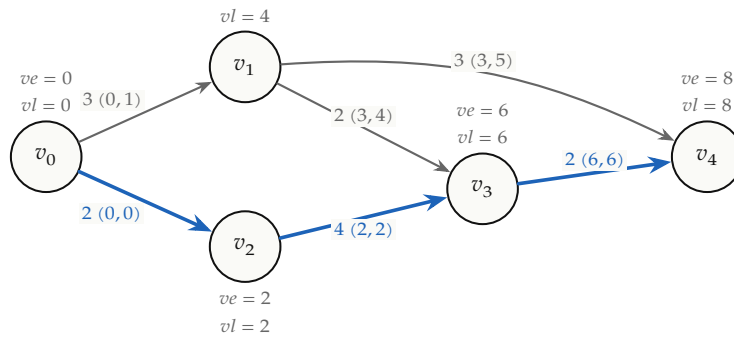
对活动 $u \rightarrow v$ ，其最早开始与最迟开始分别为

$$e = ve[u], \quad l = vl[v] - w(u, v).$$

只有 $e = l$ ，即总时差为零，活动才是关键活动。

AOE 关键路径：事件算 ve/vl ，活动算 e/l ，零时差才构成关键链

边上的前一个数是持续时间 w ；括号内给出活动 (e, l) 。蓝色实线表示 $l - e = 0$ 的关键活动。



关键路径： $v_0 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4$ ，工期 = 8。

图中纵向位置只为避让，不表示事件时间；真正时间只看 ve/vl 数值与边权。

正向：事件最早时间

$ve[v] = \max(ve[u] + w)$ 。

本例 $ve[v_3] = \max(3 + 2, 2 + 4) = 6$ 。

反向：事件最迟时间

$vl[u] = \min(vl[v] - w)$ 。

本例 $vl[v_1] = \min(6 - 2, 8 - 3) = 4$ 。

活动判定

对 $u \rightarrow v$ ： $e = ve[u]$ ， $l = vl[v] - w$ 。

只有 $e = l$ 才是关键活动。

图中节点只承载事件的 ve/vl ，边承载活动持续时间和 (e, l) ；纵向排布只为避让，不表示事件时间。

若要估算缩短工期的代价，先找当前关键路径；非关键活动有正总时差，单独缩短它不会改变项目工期。只有缩短所有当前关键路径上的至少一条活动，工期才可能下降；存在多条互不共享的关键路径时，必须同时在每条路径上选活动。活动可缩短量受工期下限或资源上限约束，不能把“零时差”误当成“可无限压缩”。

对活动网络常同时计算四个量：事件最早发生时间 ve 、事件最迟发生时间 vl ，活动最早开始时间 e 、活动最迟开始时间 l 。关键活动的判据是 $l - e = 0$ ；关键路径是从源事件到汇事件的一条零时差活动链，项目工期等于汇事件的 ve 。若有多个零时差分支，关键路径可能不止一条，不能只输出任意一条就声称唯一。正向最长路必须在拓扑序上做，反向收紧必须在逆拓扑序上做；未先验证 DAG 时，两个循环都没有定义。

为什么只算正向最长路还不够

正向一遍只能回答“项目最早多久完成”；它不能判断某条活动能否延迟而不影响总工期。关键活动判断必须依赖反向的最迟时间。代码若只返回 `earliest`，就没有完成关键路径算法。

```
1 struct CriticalPathResult {
2     Distance duration = 0;
3     std::vector<Distance> earliest_event;
4     std::vector<Distance> latest_event;
5     std::vector<std::size_t> critical_edge_indices;
6 };
7
8 inline CriticalPathResult critical_path(std::size_t vertex_count,
9                                         const std::vector<Edge>& edges) {
10     for (const Edge& edge : edges) {
11         check_edge(vertex_count, edge);
12         if (edge.weight < 0) {
13             throw std::invalid_argument("activity duration must be nonnegative");
14         }
15     }
16     const std::vector<std::size_t> order = topological_order(vertex_count, edges);
17     CriticalPathResult result;
18     result.earliest_event.assign(vertex_count, 0);
19     for (const std::size_t current : order) {
```

```

20     for (const Edge& edge : edges) {
21         if (edge.from == current) {
22             result.earliest_event[edge.to] =
23                 std::max(result.earliest_event[edge.to],
24                     result.earliest_event[current] + edge.weight);
25         }
26     }
27 }
28 for (const Distance time : result.earliest_event) {
29     result.duration = std::max(result.duration, time);
30 }
31 result.latest_event.assign(vertex_count, result.duration);
32 for (auto current = order.rbegin(); current != order.rend(); ++current) {
33     for (const Edge& edge : edges) {
34         if (edge.from == *current) {
35             result.latest_event[edge.from] =
36                 std::min(result.latest_event[edge.from],
37                     result.latest_event[edge.to] - edge.weight);
38         }
39     }
40 }
41 for (std::size_t index = 0; index < edges.size(); ++index) {
42     const Edge& edge = edges[index];
43     const Distance earliest_start = result.earliest_event[edge.from];
44     const Distance latest_start = result.latest_event[edge.to] - edge.weight;
45     if (earliest_start == latest_start) {
46         result.critical_edge_indices.push_back(index);
47     }
48 }
49 return result;
50 }

```

代码清单 7: 关键路径: 最早、最迟与零时差边

5 测试不是报喜, 而是攻击不变量

```

1 void test_mst_algorithms_share_the_same_goal() {
2     const std::vector<Edge> edges{{0, 1, 1}, {1, 2, 2}, {2, 3, 1}, {0, 3, 8}};
3     assert(prim_weight(4, edges) == 4);
4     assert(kruskal_weight(4, edges) == 4);
5     assert_invalid([&] { (void)prim_weight(5, edges); });
6     assert_invalid([&] { (void)kruskal_weight(5, edges); });
7 }
8
9 void test_shortest_path_algorithms_and_boundaries() {
10     const std::vector<Edge> nonnegative{
11         {0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}, {2, 3, 5}};
12     const std::vector<Distance> expected{0, 3, 1, 4, kInfinity};
13     assert(dijkstra(5, nonnegative, 0) == expected);
14     assert(bellman_ford(5, nonnegative, 0) == expected);
15 }

```

```

16  const std::vector<Edge> with_negative{{0, 1, 4}, {1, 2, -3}, {0, 2, 5}};
17  assert((bellman_ford(3, with_negative, 0) == std::vector<Distance>{0, 4, 1})
18         );
19  assert_invalid([&] { (void)dijkstra(3, with_negative, 0); });
20  assert_invalid([&] { (void)bellman_ford(2, {{0, 1, -1}, {1, 0, -1}}, 0); });
21
22  const auto all_pairs = floyd_warshall(4, nonnegative);
23  assert(all_pairs[0][3] == 4);
24  assert(all_pairs[3][0] == kInfinity);
25  assert_invalid([&] { (void)floyd_warshall(2, {{0, 1, -1}, {1, 0, -1}}); });
26
27  const std::vector<Edge> large_weights{{0, 1, 2'000'000'000}, {1, 2, 2'000'000
28         '000}};
29  assert(dijkstra(3, large_weights, 0)[2] == 4'000'000'000LL);
30 }
31
32 void test_topology_and_critical_path() {
33     const std::vector<Edge> activities{
34         {0, 1, 3}, {0, 2, 2}, {1, 3, 4}, {2, 3, 3}, {2, 4, 2}, {3, 5, 2}, {4, 5,
35             3}};
36     const auto order = topological_order(6, activities);
37     assert(order.front() == 0 && order.back() == 5);
38
39     const auto result = critical_path(6, activities);
40     assert(result.duration == 9);
41     assert((result.earliest_event == std::vector<Distance>{0, 3, 2, 7, 4, 9}));
42     assert((result.latest_event == std::vector<Distance>{0, 3, 4, 7, 6, 9}));
43     assert((result.critical_edge_indices == std::vector<std::size_t>{0, 2, 5}));
44
45     assert_invalid([&] { (void)topological_order(2, {{0, 1, 1}, {1, 0, 1}}); });
46     assert_invalid([&] { (void)critical_path(2, {{0, 1, -1}}); });
47 }

```

代码清单 8: 同目标对照、失败边界与完整关键路径断言

测试按容易混淆的接口成对设计：Prim 与 Kruskal 在同图上应得到同一 MST 总权；Dijkstra 与 Bellman–Ford 在非负图上应得到相同单源距离；负边只拒绝 Dijkstra，负环才拒绝 Bellman–Ford；拓扑排序输出序列，关键路径还必须输出 *ve*、*vl* 与关键边。

路径代价使用 long long，并用两条 2×10^9 的边验证结果为 4×10^9 ，避免算法逻辑正确却被 32 位整数边界破坏。

6 复杂度与适用边界

算法	本实现复杂度	正确性前提	显式失败
Prim	$O(V^2 + E)$	无向连通带权图	非连通
Kruskal	$O(E \log E)$	无向连通带权图	非连通
Dijkstra	$O((V + E) \log V)$	有向图边权非负	负边
Bellman-Ford	$O(VE)$	允许负边	源可达负环
Floyd	$O(V^3)$	距离加法不溢出	任意负环
拓扑排序	$O(V + E)$	有向无环图	有环
关键路径	当前实现 $O(VE)$	AOE 网为 DAG, 工期非负	有环、负工期

7 考场与代码复原协议

遇到图算法题，按五步写在草稿上：

1. **Goal:** 要树、单源距离、全源距离、合法顺序，还是总工期与关键活动？
2. **Candidate:** 候选是跨割边、全局边、暂定距离、全部边、点对，还是零入度点？
3. **Commit:** 本轮结束后，哪个对象从“候选”变为“永久成立”？
4. **Update:** 更新距离、入度、分量、跨边权，还是最早/最迟时间？
5. **Failure:** 负边、负环、非连通、有环、不可达与溢出分别怎样显式出现？

压缩复原

只记五问：求什么？候选是谁？何时提交？提交后什么永真？什么证据让证明失效？当五问能在新图上独立回答，代码才真正被压缩成心智模型；只会背函数名和循环层数，还没有形成模型。

代码证据

完整实现位于 `code/ds08_graph_algorithms.hpp`，断言测试位于 `code/ds08_graph_algorithms_test.cpp`。正文中的行号引用与当前 C++17 实现同步。